

Simulación del método de Montecarlo aplicada a una ruleta discreta

Juan Camilo Lozano Cortes, Brayan Camilo Moreno Cordero, Daniel Ayala

Guzmán y Julio Cesar Flórez Guarnizo

Universidad Sergio Arboleda



Ciencias de la Computación e desarrollo de I.A.

Asignatura: Ciencia Computacional Avanzada

Docente: Jose Marcial Tellez Gomez

Fecha: 10/02/2026

Resumen

El presente trabajo describe el desarrollo de una aplicación web orientada a la simulación del método de Montecarlo aplicado a un sistema discreto tipo ruleta. La aplicación permite definir un número arbitrario de opciones equiprobables, calcular probabilidades individuales y acumuladas, establecer intervalos de selección y aplicar el método de la función inversa para determinar el resultado de un evento aleatorio. La solución fue implementada utilizando tecnologías web estándar y modelada mediante diagramas UML, permitiendo evidenciar de forma clara el proceso de simulación estocástica.

Palabras clave: método de Montecarlo, simulación, probabilidad, función inversa, ruleta.

Introducción

El método de Montecarlo es una técnica de simulación ampliamente utilizada para modelar fenómenos que presentan incertidumbre o comportamiento aleatorio. Su principio fundamental consiste en el uso de números pseudoaleatorios para aproximar soluciones a problemas probabilísticos que resultan complejos de resolver de manera analítica. En el contexto académico, este método permite comprender de forma práctica la relación entre distribuciones de probabilidad, funciones acumuladas e intervalos de selección.

En este proyecto se desarrolla una simulación del método de Montecarlo aplicada a una ruleta discreta, donde cada opción posee la misma probabilidad de ocurrencia. La aplicación no solo construye la tabla de probabilidades, sino que implementa el método de la función inversa para determinar el resultado de una simulación a partir de un número pseudoaleatorio en el intervalo $[0,1]$.

Marco teórico

El método de Montecarlo se fundamenta en la generación de números pseudoaleatorios con distribución uniforme. A partir de estos valores, se simulan eventos que siguen una distribución de probabilidad previamente definida. En el caso de sistemas discretos, la probabilidad acumulada permite construir intervalos que representan cada posible evento.

El método de la función inversa establece que, dado un número aleatorio r perteneciente al intervalo $[0,1]$, el evento seleccionado será aquel cuyo intervalo acumulado

contenga dicho valor. Este procedimiento garantiza que la frecuencia de ocurrencia de cada evento converge a su probabilidad teórica conforme aumenta el número de simulaciones.

Metodología

La metodología empleada se divide en dos etapas principales. En la primera etapa, el usuario define el número de opciones de la ruleta. Con este valor, la aplicación calcula la probabilidad individual, la probabilidad acumulada y los intervalos inicial y finales asociados a cada opción.

En la segunda etapa, el sistema genera un número pseudoaleatorio utilizando una función uniforme o permite el ingreso manual de un valor entre cero y uno. Dicho valor es comparado con los intervalos definidos previamente para determinar la opción seleccionada, siguiendo el método de la función inversa.

Implementación del sistema.

La aplicación fue desarrollada utilizando HTML para la estructura de la interfaz, CSS para el diseño visual y JavaScript para la lógica de simulación. El sistema se organiza conceptualmente en una clase principal denominada Simulación Montecarlo, la cual gestiona el proceso general y coordina la generación de probabilidades, el dibujo de la ruleta y la simulación del evento aleatorio.

Diagramas UML

Diagrama de clases de la simulación Montecarlo

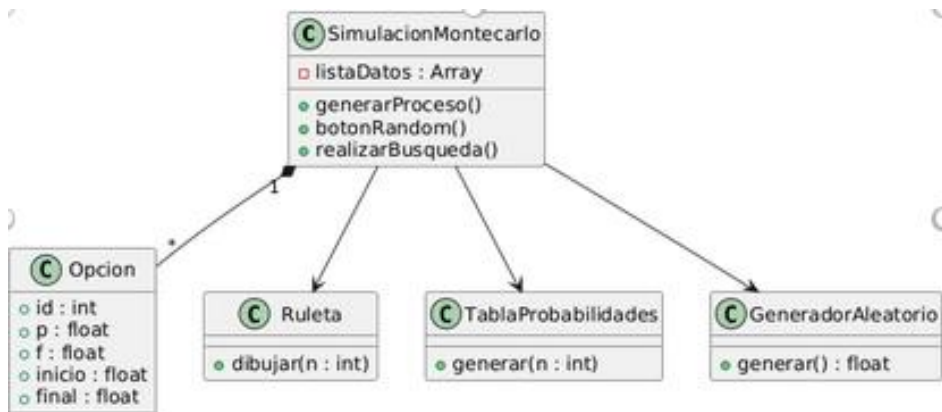
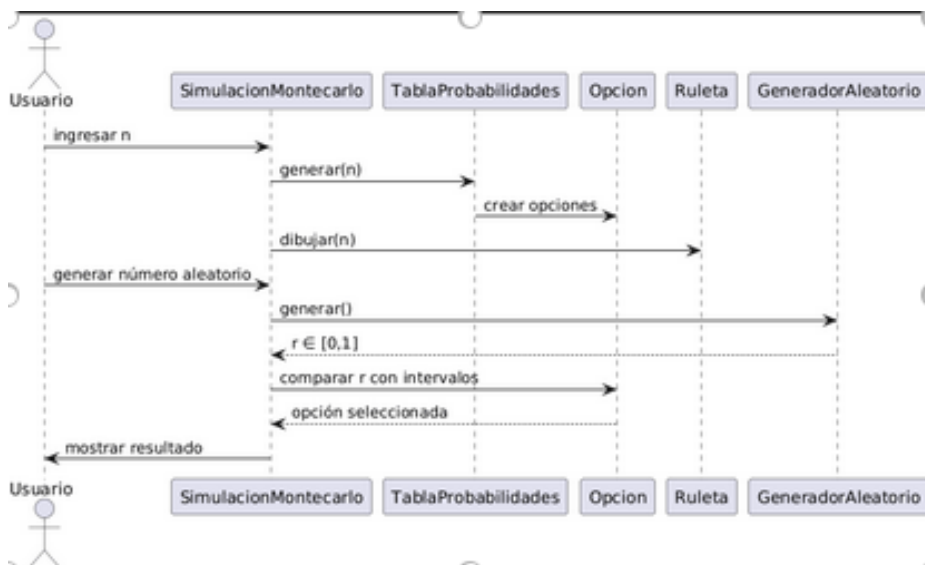


Diagrama de colaboración del proceso de simulación



Código

<!DOCTYPE html>

```

<html lang="es">

<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Simulación Montecarlo</title>
<link rel="stylesheet" href="styles.css" />
</head>

<body>

<header class="topbar">
<div class="topbar__inner topbar__inner--simple">
<h1 class="topbar__title">Simulación Método de Montecarlo</h1>
</div>
</header>

<main class="page">
<section class="card card--authors">
<h2 class="card__title">Integrantes</h2>
<div class="authors">
<span class="chip">Juan Camilo Lozano Cortes</span>
<span class="chip">Brayan Camilo Moreno Cordero</span>
<span class="chip">Daniel Ayala Guzman</span>
<span class="chip">Julio Cesar Florez Guarnizo</span>
</div>
</section>

<section class="grid">
#PASO 1
<section class="card">
<div class="card__head">
<h2 class="card__title">Paso 1 · Configurar opciones</h2>
<p class="card__desc">Define cuántas opciones tendrá la simulación y genera la
tabla + ruleta.</p>
</div>

<div class="controls">
<div class="field">
<label for="inputN">Cantidad de opciones (n)</label>
<div class="field__row">
<input type="number" id="inputN" value="8" min="2" max="200" />

```

```
<button class="btn btn--primary" id="btnGenerar" type="button">Generar</button>
</div>
<p class="help">Recomendado: 6-20 para visualizar mejor la ruleta.</p>
</div>
```

```
<div class="ruletaWrap" aria-label="Ruleta de opciones">
  <div class="ruleta">
    <svg id="miRuleta" width="240" height="240" viewBox="0 0 32 32" role="img"
aria-label="Ruleta"></svg>
    <div class="ruleta__pin" aria-hidden="true"></div>
  </div>
```

```
<div class="ruletaInfo">
  <div class="stat">
    <span class="stat__label">Opciones</span>
    <span class="stat__value" id="statN">8</span>
  </div>
  <div class="stat">
    <span class="stat__label">p(X)</span>
    <span class="stat__value" id="statP">0.125</span>
  </div>
  <div class="legend" id="legend"></div>
</div>
</div>
</div>
```

```
<div class="tableWrap" role="region" aria-label="Tabla de probabilidades">
  <table class="table">
    <thead>
      <tr>
        <th>Opción X</th>
        <th>p(X)</th>
        <th>F(X) (Acumulada)</th>
        <th>Intervalo Inicial</th>
        <th>Intervalo Final</th>
      </tr>
    </thead>
    <tbody id="tablaCuerpo"></tbody>
  </table>
</div>
</section>
```

```

#PASO 2
<section class="card">
  <div class="card__head">
    <h2 class="card__title">Paso 2 · Simulación</h2>
    <p class="card__desc">La ruleta gira y se elige automáticamente una opción (y un
número r dentro del
intervalo).</p>
  </div>

  <div class="controls">
    <div class="actions">
      <button class="btn btn--primary" id="btnGirarYSimular" type="button">Girar y
simular</button>
    </div>

    #Opcional: muestra el r generado (solo lectura)
    <div class="field field--inline">
      <label for="inputR">Número generado (0-1)</label>
      <input type="number" id="inputR" step="0.001" min="0" max="1" readonly />
    </div>

    <div class="result" id="cuadroResultado" role="status" aria-live="polite" aria-
atomic="true"></div>

    <div class="note">
      <p><strong>Nota:</strong> si cambias <em>n</em>, vuelve a presionar
<em>Generar</em> antes de simular.</p>
    </div>
  </div>
</section>
</section>
</main>

<footer class="footer" aria-hidden="true"></footer>

<script>
let listaDatos = [];
let lastSelected = null;
let isSpinning = false;
let currentRotation = -90;

const colores = [

```



```
"#2563EB", "#10B981", "#F59E0B", "#EF4444", "#8B5CF6", "#06B6D4",  
"#F97316", "#22C55E", "#A855F7", "#0EA5E9", "#E11D48", "#84CC16"  
];
```

```
const $ = (id) => document.getElementById(id);  
const fmt3 = (x) => Number(x).toFixed(3);
```

```
function clamp01(x) { return Math.max(0, Math.min(1, x)); }
```

```
function setResult(html, kind = "ok") {  
  const res = $("cuadroResultado");  
  res.classList.remove("result--ok", "result--warn");  
  res.classList.add(kind === "warn" ? "result--warn" : "result--ok");  
  res.innerHTML = html;  
  res.style.display = "block";  
}  
function clearResult() {  
  const res = $("cuadroResultado");  
  res.innerHTML = "";  
  res.style.display = "none";  
  res.classList.remove("result--ok", "result--warn");  
}
```

```
function setWheelRotation(deg) {  
  $("miRuleta").style.setProperty("--wheel-rot", `${deg}deg`);  
}
```

```
function generarProceso() {  
  const nRaw = parseInt($("#inputN").value, 10);  
  const n = Number.isFinite(nRaw) ? nRaw : 0;
```

```
  if (n < 2 || n > 200) {  
    setResult("<strong>Error:</strong> n debe estar entre 2 y 200.", "warn");  
    return;  
  }
```

```
  clearResult();  
  isSpinning = false;
```

```
  const cuerpo = $("tablaCuerpo");  
  const grafico = $("miRuleta");  
  const legend = $("legend");
```

```

cuerpo.innerHTML = "";
grafico.innerHTML = "";
legend.innerHTML = "";
listaDatos = [];
lastSelected = null;

const prob = 1 / n;
let acum = 0;

$("statN").textContent = String(n);
$("statP").textContent = fmt3(prob);

const porcentajeCorte = (1 / n) * 100;

for (let i = 0; i < n; i++) {
  const ini = acum;
  acum += prob;
  if (i === n - 1) acum = 1.0;

  const fila = {
    id: i,
    p: fmt3(prob),
    f: fmt3(acum),
    inicio: fmt3(ini),
    final: fmt3(acum),
    color: colores[i % colores.length]
  };
  listaDatos.push(fila);

  const tr = document.createElement("tr");
  tr.dataset.id = String(i);
  tr.innerHTML = `
    <td><span class="opt"><span class="swatch" style="--
sw:${fila.color}"></span>Opción ${fila.id}</span></td>
    <td>${fila.p}</td>
    <td>${fila.f}</td>
    <td>${fila.inicio}</td>
    <td>${fila.final}</td>
  `;
  cuerpo.appendChild(tr);

```

```

const posicionCorte = (i / n) * 100;
const circle = document.createElementNS("http://www.w3.org/2000/svg", "circle");
circle.setAttribute("r", "16");
circle.setAttribute("cx", "16");
circle.setAttribute("cy", "16");
circle.setAttribute("fill", "transparent");
circle.setAttribute("stroke", fila.color);
circle.setAttribute("stroke-width", "32");
circle.setAttribute("stroke-dasharray", `${porcentajeCorte} 100`);
circle.setAttribute("stroke-dashoffset", `-${posicionCorte}`);
circle.dataset.id = String(i);
grafico.appendChild(circle);

const item = document.createElement("button");
item.type = "button";
item.className = "legend__item";
item.innerHTML = `<span class="swatch" style="--
sw:${fila.color}"></span><span>Opción ${i}</span>`;
item.addEventListener("click", () => highlightOption(i));
legend.appendChild(item);
}

cuerpo.querySelectorAll("tr").forEach(tr => {
tr.addEventListener("click", () => highlightOption(parseInt(tr.dataset.id, 10)));
});

currentRotation = -90;
setWheelRotation(currentRotation);

$("inputR").value = "";
}

function highlightOption(id) {
lastSelected = id;

document.querySelectorAll("#tablaCuerpo tr").forEach(tr => {
tr.classList.toggle("is-selected", parseInt(tr.dataset.id, 10) === id);
});
document.querySelectorAll("#miRuleta circle").forEach(c => {
c.classList.toggle("is-selected", parseInt(c.dataset.id, 10) === id);
});
document.querySelectorAll("#legend .legend__item").forEach((b, idx) => {

```

```

b.classList.toggle("is-selected", idx === id);
});
}

function opcionBajoPin() {
const n = listaDatos.length || 1;
const rot = ((currentRotation % 360) + 360) % 360;
const upAngle = ((90 - rot) % 360 + 360) % 360;
const sectorSize = 360 / n;
return Math.floor(upAngle / sectorSize);
}

function girarYSimilar() {
if (!listaDatos.length) generarProceso();
if (isSpinning) return;

isSpinning = true;
clearResult();

const vueltas = 4 + Math.random() * 3;
const extra = Math.random() * 360;
const targetRotation = currentRotation + (vueltas * 360) + extra;

const wheel = $("miRuleta");
wheel.classList.add("is-spinning");
void wheel.offsetWidth;

currentRotation = targetRotation;
setWheelRotation(currentRotation);

const onEnd = () => {
wheel.classList.remove("is-spinning");
wheel.removeEventListener("transitionend", onEnd);
isSpinning = false;

const idx = opcionBajoPin();
highlightOption(idx);

// r dentro del intervalo de la opción elegida
const li = Number(listaDatos[idx].inicio);
const ls = Number(listaDatos[idx].final);
const r = li + Math.random() * Math.max(0, (ls - li) - 1e-9);

```

```

$("inputR").value = fmt3(clamp01(r));

const color = listaDatos[idx]?.color ?? "#22c55e";
setResult(
  `<div class="result__row">
    <div class="badge" style="--b:${color}">Opción ${idx}</div>
    <div>
      <div><strong>Número generado:</strong> ${fmt3(r)}</div>
      <div class="muted">Intervalo: [${listaDatos[idx].inicio},
${listaDatos[idx].final}]</div>
    </div>
  </div>`,
  "ok"
);
};

wheel.addEventListener("transitionend", onEnd);
}

// Eventos
$("btnGenerar").addEventListener("click", generarProceso);
$("btnGirarYSimilar").addEventListener("click", girarYSimilar);

$("inputN").addEventListener("keydown", (e) => {
  if (e.key === "Enter") generarProceso();
});

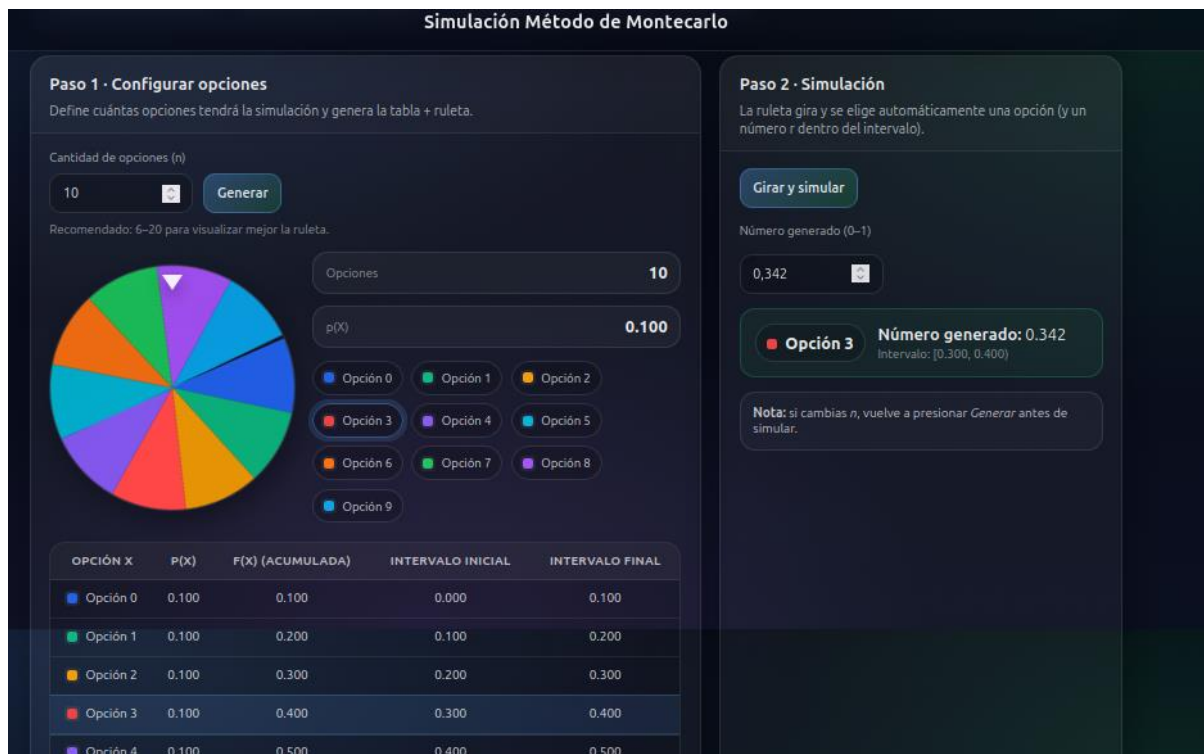
// Inicial
generarProceso();
</script>

</body>

</html>

```

Vista



Resultados

La aplicación permite observar de forma clara el comportamiento del método de Montecarlo. Para cualquier número de opciones definido por el usuario, el sistema genera correctamente la tabla de probabilidades y asigna de manera consistente los intervalos de selección. Al introducir un número pseudoaleatorio, el sistema identifica de forma precisa la opción correspondiente, evidenciando la correcta implementación del método de la función inversa.

Conclusiones

La simulación desarrollada cumple con los objetivos planteados, demostrando la aplicabilidad del método de Montecarlo en sistemas discretos. La representación tabular, visual y algorítmica facilita la comprensión del proceso probabilístico y refuerza los conceptos teóricos estudiados.

Referencias

Banks, J., Carson, J. S., Nelson, B. L., y Nicol, D. M. (2010). Discrete-event system simulation. Pearson.

Ross, S. M. (2014). Introduction to probability models (11.a ed.). Academic Press.

American Psychological Association. (2020). Style and Grammar Guidelines.

<https://apastyle.apa.org/style-grammar-guidelines>